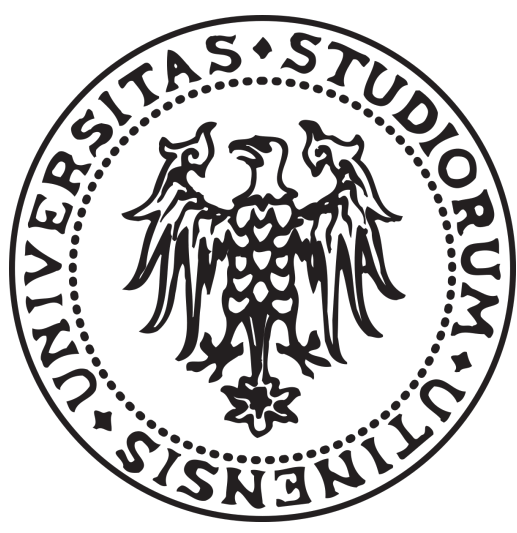




INGEGNERIA DEL SOFTWARE - 2025-26

TESTING: CONCETTI FONDAMENTALI

LEZIONE 23
17/12/2025
VINCENZO RICCIO

RIFERIMENTI

- Sommerville - Capitolo 8

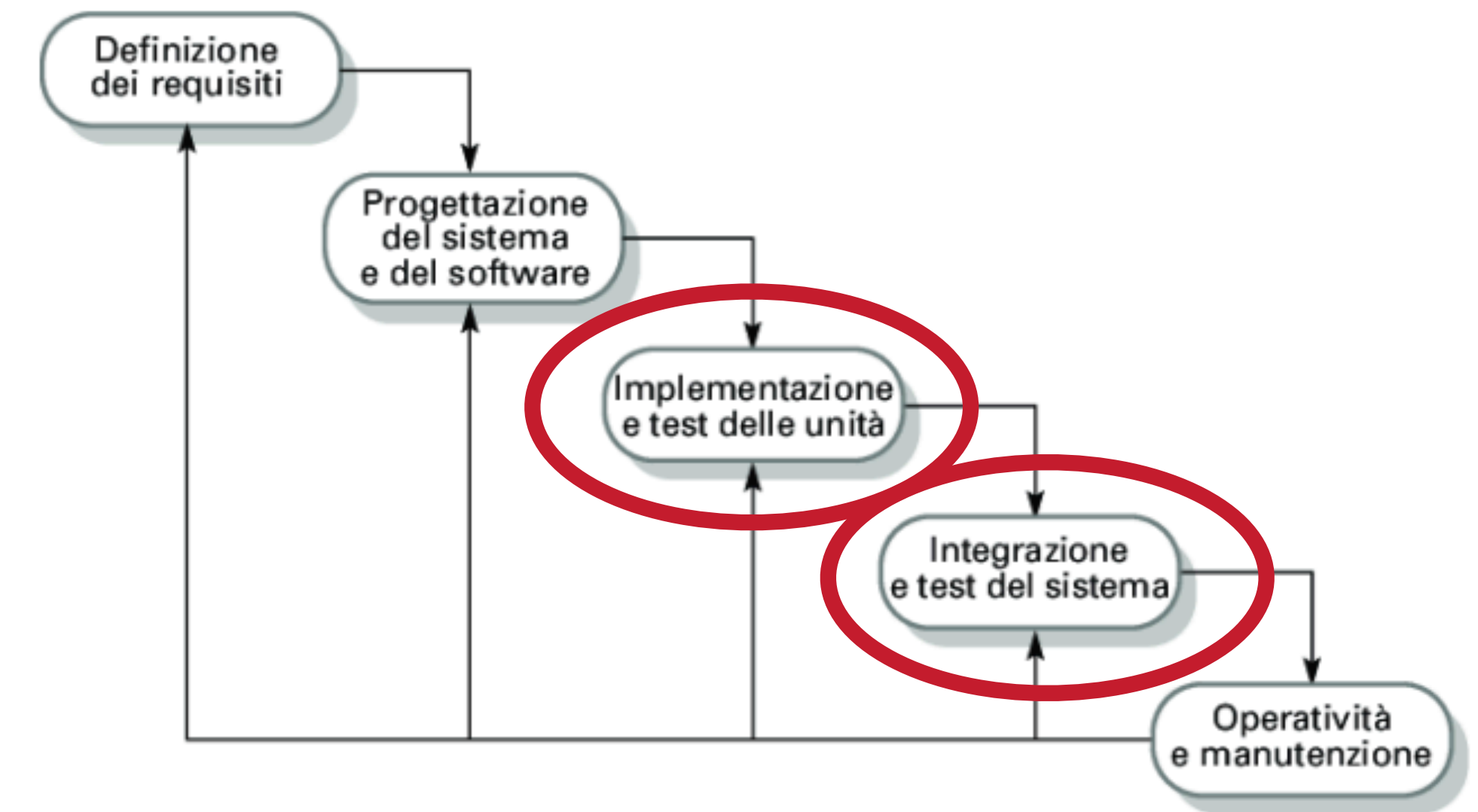
Vincenzo Riccio
Ingegneria del Software 2025/2026
Università degli Studi di Udine



ATTIVITÀ DI TESTING NEL PROCESSO SOFTWARE



- Insieme di attività che hanno il compito di dimostrare che un programma svolge i compiti per i quali è stato realizzato, rilevando eventuali difetti prima di metterlo in operatività





▸ **Verifica (Verification)**

- ◎ *Are we building the software right?*
- ◎ Controllare che il software soddisfi la specifica dei requisiti e gli standard stabiliti all'inizio del ciclo di sviluppo

▸ **Validazione (Validation)**

- ◎ *Are we building the right software?*
- ◎ Dimostrare che il software risolva il giusto problema, supportando l'utilizzo atteso e soddisfacendo i bisogni degli utenti nell'ambiente operativo

TESTING E DEBUGGING



- L'attività di testing rientra nel processo più ampio di Verifica e Validazione (V&V)
- Il **testing** (collaudo) è un processo di esecuzione del software allo scopo di scoprirne eventuali malfunzionamenti
- La presenza di malfunzionamenti indica la presenza di difetti nel software





ERRORE, DIFETTO, MALFUNZIONAMENTO

ERRORE (ERROR)



- Situazione frutto di incomprensione umana (da parte del singolo sviluppatore o del team di sviluppo)
- Esistono molte tipologie di errori, che possono essere commessi
 - Nel tentativo di comprendere i requisiti del problema
 - Nell'elaborare una soluzione al problema
 - Nel comprendere o utilizzare strumenti
 - Nella programmazione, dovuti a distrazione, superficialità o problemi di comunicazione

DIFETTO (FAULT)



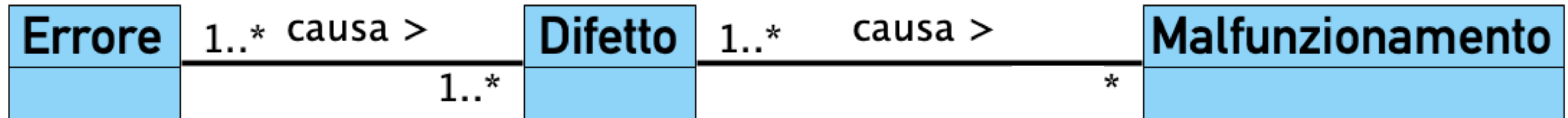
- Manifestazione nel software di un errore umano
 - Il difetto ha una natura statica, ossia consiste in codice sintatticamente o semanticamente sbagliato
 - Se il codice difettoso non viene mai esercitato, potrebbe passare inosservato poiché non scatenerrebbe nessun malfunzionamento del software durante l'esecuzione

MALFUNZIONAMENTO (FAILURE)



- Incapacità del software di comportarsi secondo le aspettative o le specifiche
 - ◎ Il malfunzionamento ha una natura dinamica, accade solo in un certo istante di tempo e può essere osservato solo mediante esecuzione
 - ◎ Il malfunzionamento più estremo e ovvio è il crash, altri possono essere meno evidenti

ERRORE, DIFETTO E MALFUNZIONAMENTO



```
Function raddoppia (x)
...
read(x)
y = x * x
return y
```

```
> raddoppia (2)
> 4
> raddoppia (3)
> 9
```

- Il difetto é causato da uno o più errori e **può causare** uno o più malfunzionamenti
- **Errore**: di digitazione
- **Difetto**: * invece di +
- **Malfunzionamento**: durante l'esecuzione il risultato visualizzato é diverso da quello atteso



CASI DI TEST



- Durante il testing, il software è esercitato da un insieme di input e ne viene valutato il comportamento
- Un caso di test è particolarmente utile se è in grado di scoprire un malfunzionamento non ancora scoperto da altri casi di test
- Pertanto non basta definire gli input di ciascun caso di test (**test case**), ma bisogna definire anche l'output, le pre-condizioni e post-condizioni

ID	Descrizione	Precondizioni	Valori di Input	Output atteso	Output Effettivo	Postcondizioni attese	Postcondizioni effettive	Esito
----	-------------	---------------	-----------------	---------------	------------------	-----------------------	--------------------------	-------



- L'identificativo ID aiuta a specificare e comunicare il caso di test
- La descrizione può indicare il requisito che il caso di test intende testare
- I valori di input indicano il modo in cui il programma sarà testato
- Le precondizioni sono condizioni che devono essere verificate affinché il caso di test venga eseguito

ID	Descrizione	Precondizioni	Valori di Input	Output atteso	Output Effettivo	Postcondizioni attese	Postcondizioni effettive	Esito
----	-------------	---------------	-----------------	---------------	------------------	-----------------------	--------------------------	-------



- I valori di output attesi rappresentano il comportamento atteso dal software dopo essere stato esercitato dagli input specificati
- Le postcondizioni attese rappresentano le condizioni in cui si dovrebbe trovare il sistema dopo l'esecuzione del caso di test

ID	Descrizione	Precondizioni	Valori di Input	Output atteso	Output Effettivo	Postcondizioni attese	Postcondizioni effettive	Esito
----	-------------	---------------	-----------------	---------------	------------------	-----------------------	--------------------------	-------



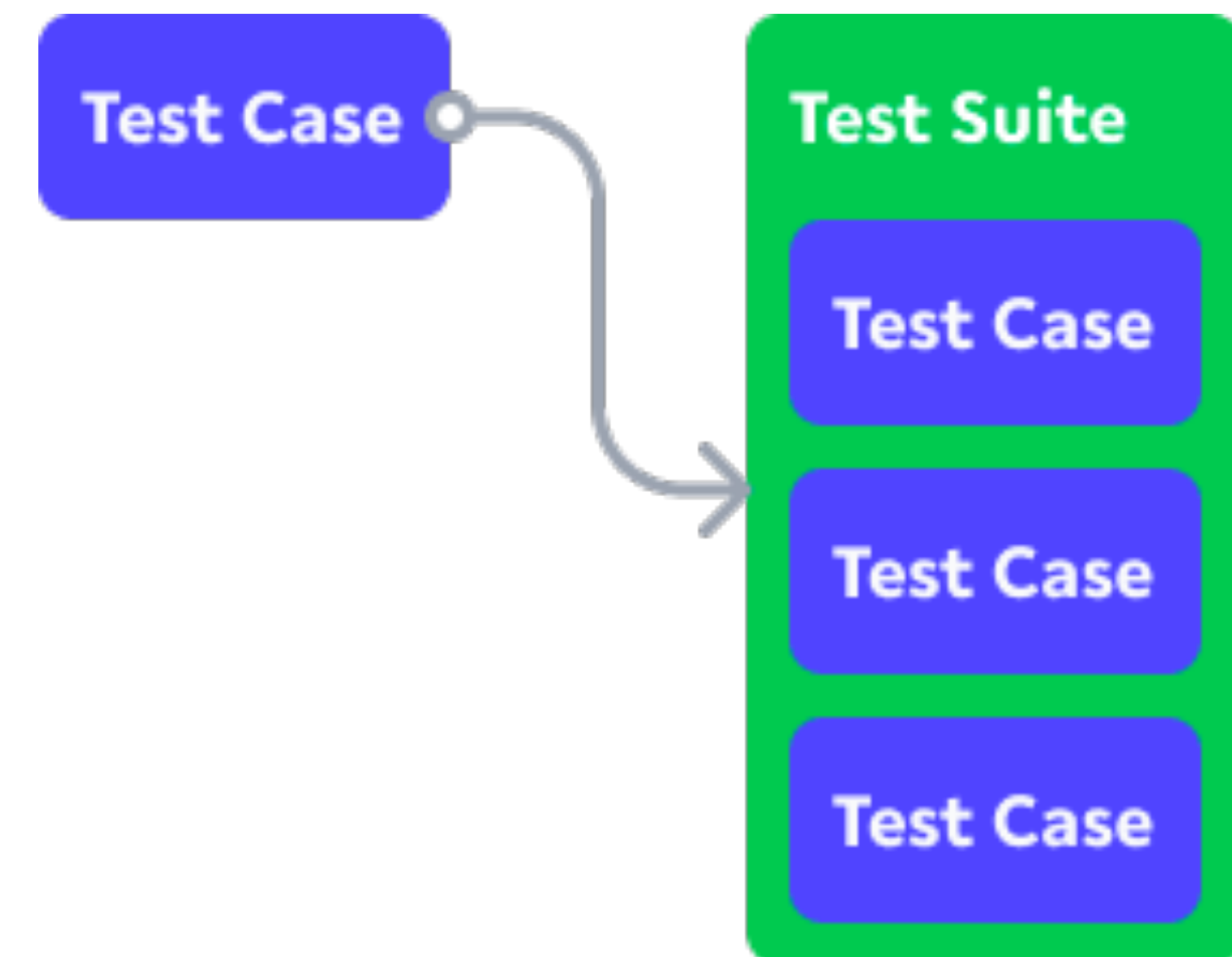
- Dopo l'esecuzione del caso di test è possibile conoscere l'output effettivo e le postcondizioni effettive
- L'esito sarà:
 - ◎ **Malfunzionamento rilevato** se almeno un valore di output o una postcondizione riscontrati dopo l'esecuzione del caso di test sono diversi da quelli attesi
 - ◎ **Malfunzionamento non rilevato** altrimenti

ID	Descrizione	Precondizioni	Valori di Input	Output atteso	Output Effettivo	Postcondizioni attese	Postcondizioni effettive	Esito
----	-------------	---------------	-----------------	---------------	------------------	-----------------------	--------------------------	-------

TEST SUITE



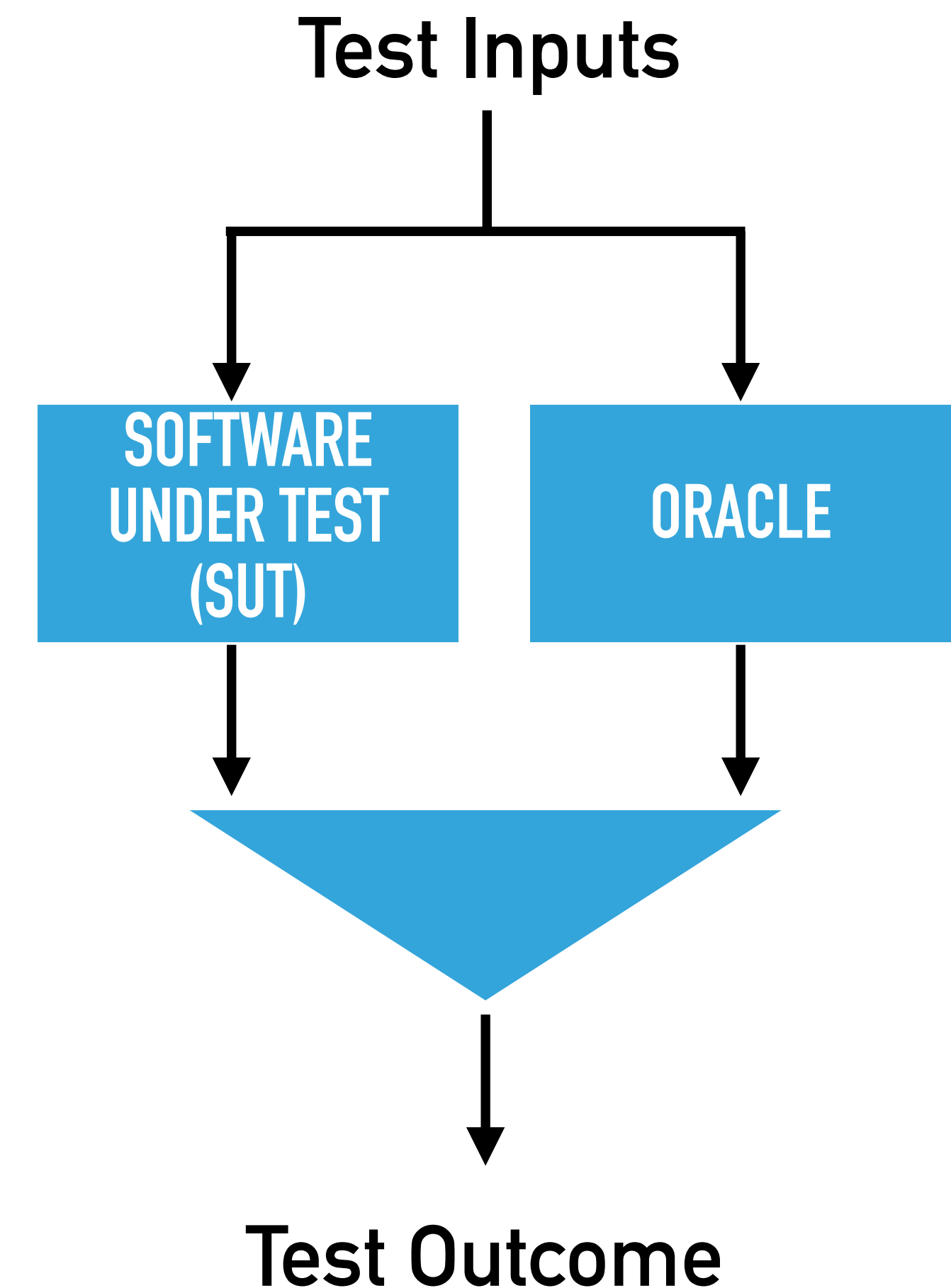
- Un caso di test TC_i rileva la presenza di uno o più malfunzionamenti se il programma P non è corretto per TC_i ovvero se produce risultati o postcondizioni diversi da quelli attesi
- Un insieme di casi di test detto **test suite** mira a rilevare la presenza di più difetti possibili
- Bisogna selezionare in maniera appropriata i casi di test appartenenti alla test suite



ORACOLO



- L'oracolo conosce il comportamento atteso per ogni caso di test
- Permette di confrontare il risultato atteso con quello effettivo
- Diversi tipi di oracolo:
 - Oracolo umano
 - Oracolo automatico che può essere (1) generato da specifiche formali, (2) software con le stesse funzionalità ma sviluppato da altri, (3) versione precedente del software [test di regressione]



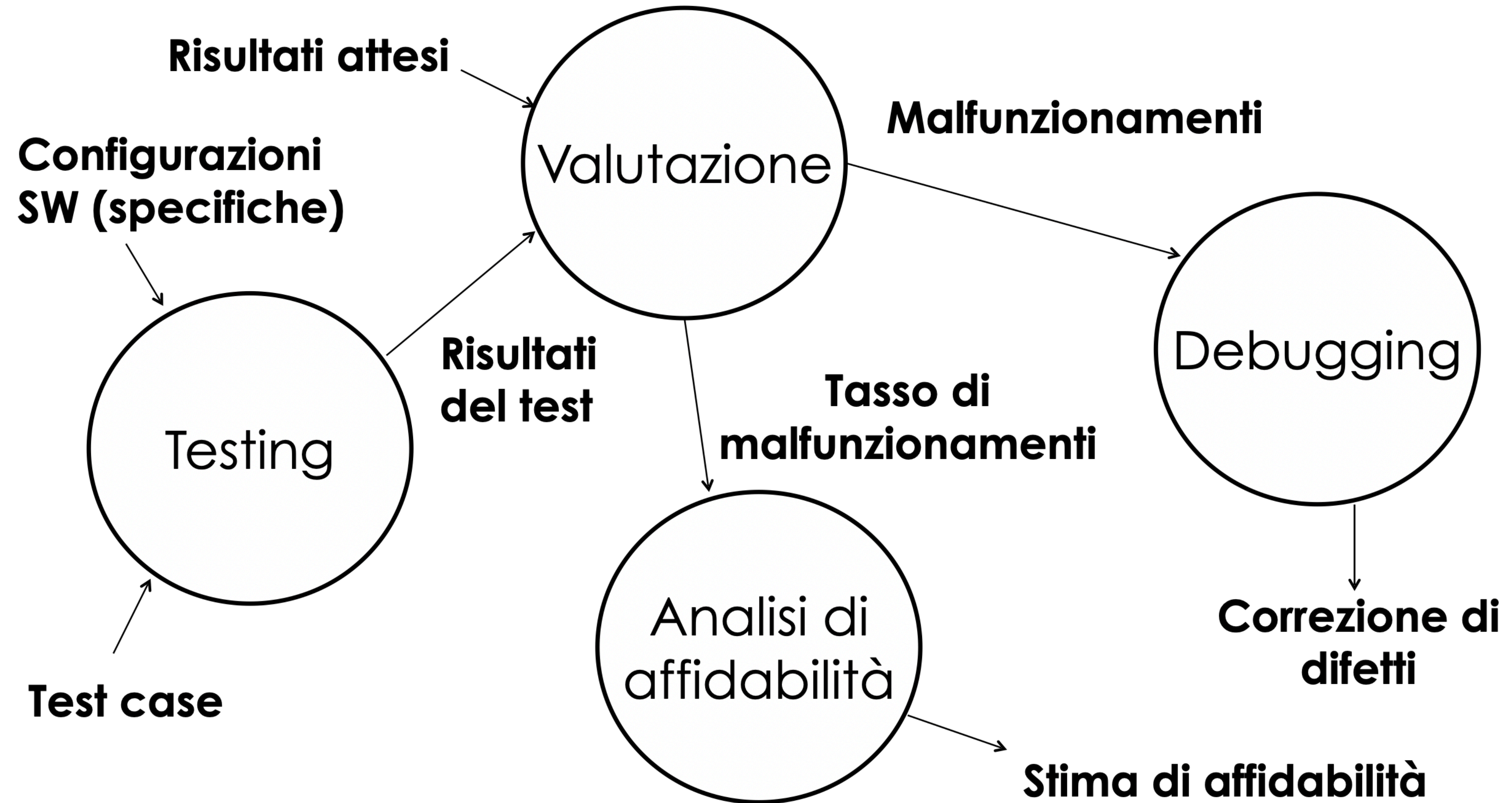


OBIETTIVI DEL TESTING



- Il testing rileva la presenza di malfunzionamenti, non identifica i difetti
- Il **debugging** è il processo di scoperta dei difetti a partire dai malfunzionamenti rilevati. Si occupa di identificare e rimuovere le cause di un malfunzionamento
- L'**analisi dell'affidabilità** si occupa di fornire una stima della probabilità che il software operi senza malfunzionamenti per un determinato periodo di tempo e in uno specifico ambiente
- La rilevazione con una certa regolarità di malfunzionamenti gravi, che richiedono pesanti modifiche alla progettazione, mette in discussione l'affidabilità e la qualità del software

TESTING, DEBUGGING E ANALISI DI AFFIDABILITÀ





- Il testing è **ideale** se l'assenza di malfunzionamenti rilevati implica la correttezza del programma
- Il testing è **esaustivo** se contiene tutte le possibili combinazioni dei dati di ingresso al programma
 - Un test esaustivo è ideale
- In casi non banali, il test esaustivo non è pratico ed è infattibile a causa di costi elevatissimi
- Un obiettivo realistico è selezionare casi di test (**test selection**) che approssimino un test ideale e diano fiducia sulla qualità del software



- La bontà di una test suite si può misurare in termini di
- **Efficacia:** $\# \text{ malfunzionamenti trovati} / \# \text{ malfunzionamenti da trovare}$
 - In realtà non è quasi mai calcolabile poiché non si conosce il numero totale di malfunzionamenti
 - Da privilegiare quando si vuole software affidabile
- **Efficienza:** $\# \text{ test che rivelano malfunzionamenti} / \# \text{ test totali}$
 - Da privilegiare quando si vuole un testing meno costoso
- Idealmente bisognerebbe trovare il maggior numero possibile di malfunzionamenti con il minor numero possibile di casi di test



"I test possono dimostrare soltanto la presenza di errori, non la loro assenza."

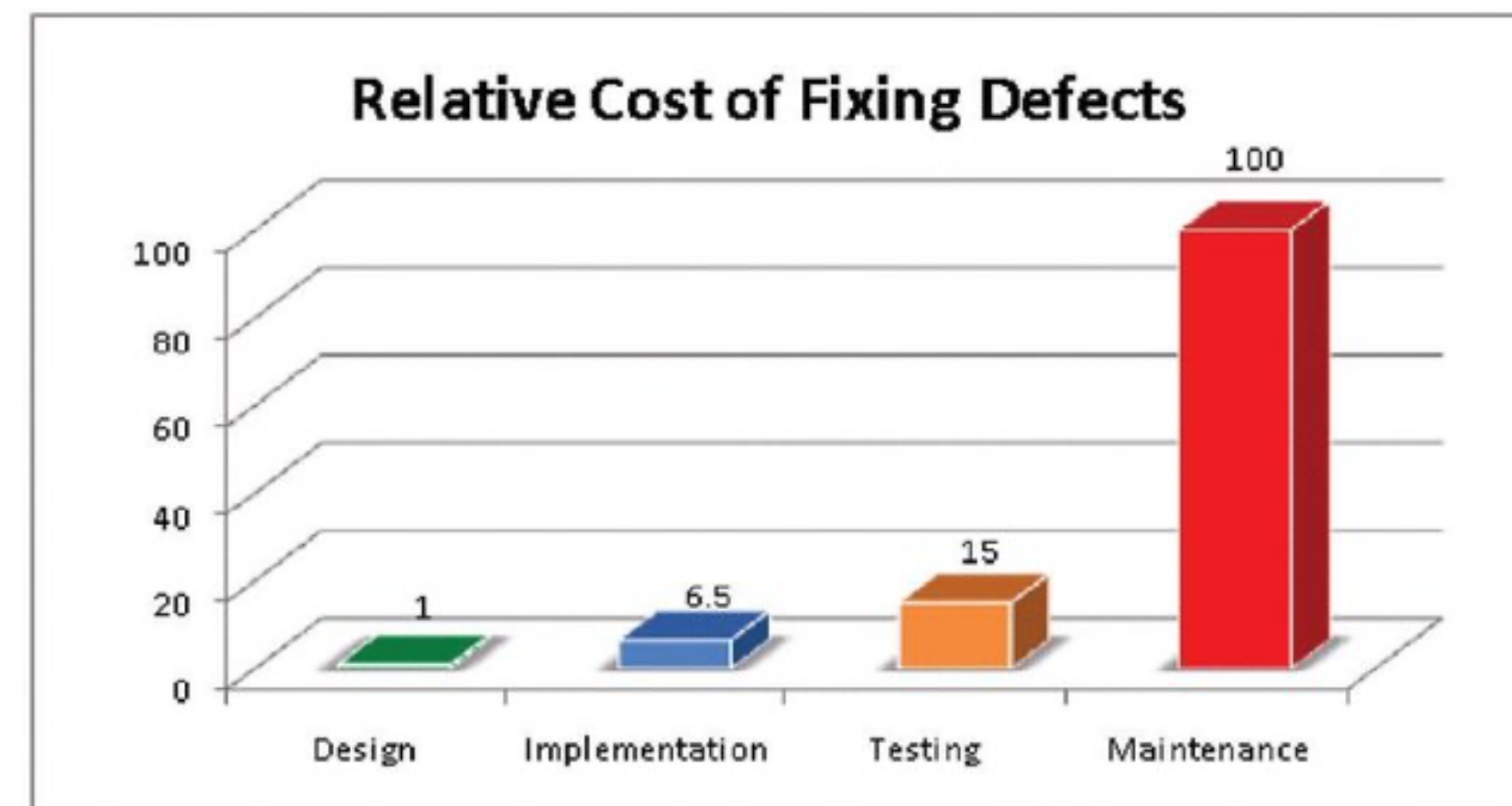
[Dijkstra, E. W. "The Humble Programmer." Comm. ACM. 1972]

- Pertanto il testing non può dimostrare la correttezza del software
- La correttezza di un programma è un problema indecidibile
- Il processo di testing deve fornire fiducia nel software, mostrando che esso è pronto per l'uso operativo

TESTING E MALFUNZIONAMENTI



- Un processo di testing che determina l'assenza di malfunzionamenti del software può significare allo stesso tempo:
 - Un'alta qualità del software
 - Inadeguatezza dei casi di test, poiché non sono stati in grado di scoprire difetti esistenti
- I malfunzionamenti non rilevati da un testing inadeguato saranno poi riscontrati dagli utenti e ciò comporta costi più elevati



CRITERI DI TERMINAZIONE E ADEGUATEZZA DEL TESTING



- Essendo il testing esaustivo non fattibile, bisogna stabilire dei criteri per decidere quando il testing possa essere terminato e ritenuto adeguato
 - **Criterio temporale:** periodo di tempo predefinito
 - **Criterio di costo:** sforzo allocato predefinito
 - **Criterio di copertura:** sono stati esercitati (coperti) un numero predefinito di obiettivi (linee di codice, scenari descritti nei casi d'uso,...)
 - **Criterio statistico:** gli ultimi k test cases non hanno rilevato malfunzionamenti



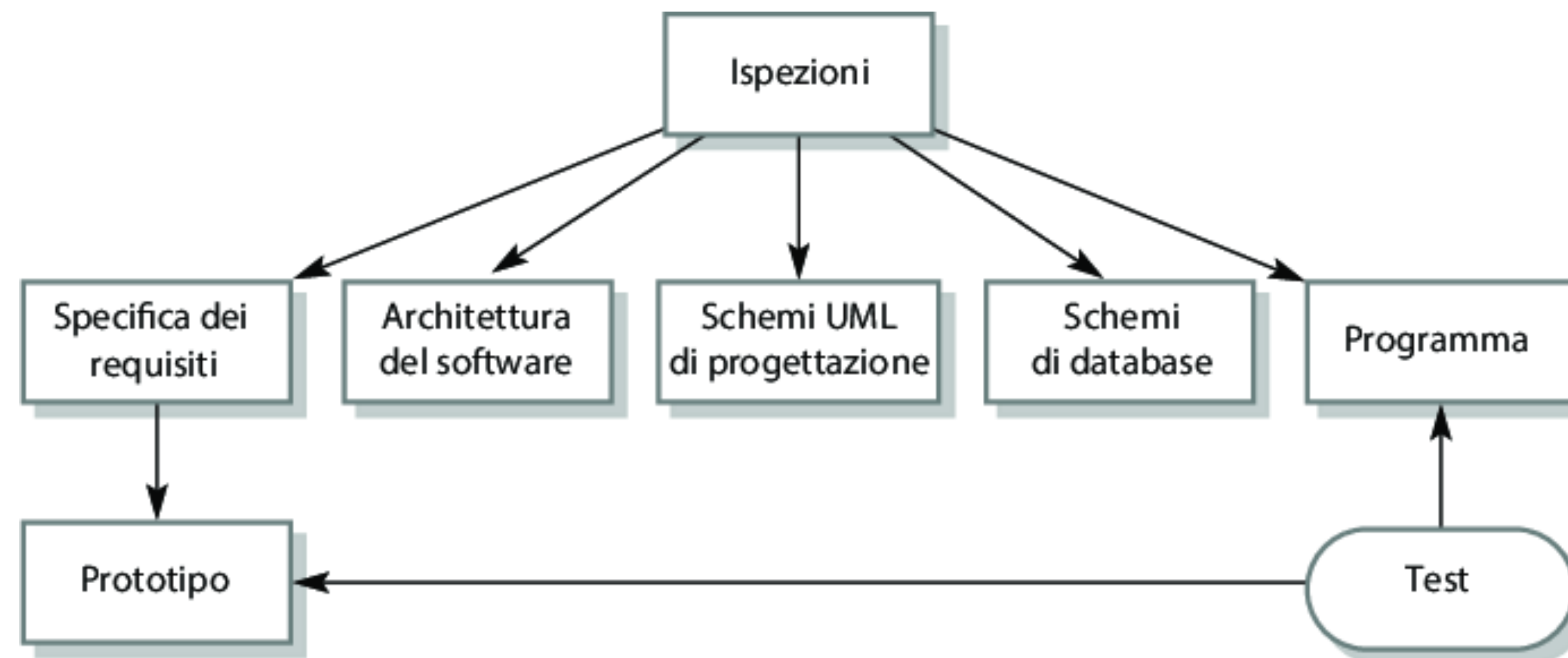
TESTING VS ISPEZIONE



- Testing ed ispezione sono due approcci complementari al problema di V&V del software
- Il testing è un'analisi **dinamica**
 - ⊙ Mira a rilevare malfunzionamenti di un software mediante l'osservazione del suo comportamento e il confronto con il comportamento atteso
 - ⊙ Il testing è possibile solo per software che possa essere eseguito
- L'ispezione è un'analisi **statica**
 - ⊙ Mira alla verifica della correttezza del software senza eseguirlo, ad es. lettura del codice
 - ⊙ L'ispezione è possibile anche per software ancora incompleto o per algoritmi che non sono stati ancora implementati



- Prende in esame i documenti, incluse le istruzioni del programma (che sono un insieme finito) senza eseguirle
- Difficile rilevare malfunzionamenti che dipendono dal valore assunto dinamicamente dalle variabili (che possono essere invece individuati dal testing)





► **Vantaggi**

- Malfunzionamenti possono mascherarne (nascondere) altri durante l'esecuzione dinamica
- Possono essere ispezionate versioni incomplete di un sistema senza costi aggiuntivi
- Oltre a cercare i difetti di un programma, un'ispezione può anche valutare attributi di qualità più generali, come la conformità agli standard e la portabilità

► **Svantaggi**

- L'analisi statica può verificare la conformità con le specifiche ma non quella con le necessità degli utenti
- L'analisi statica è meno adatta a verificare proprietà non funzionali come usabilità

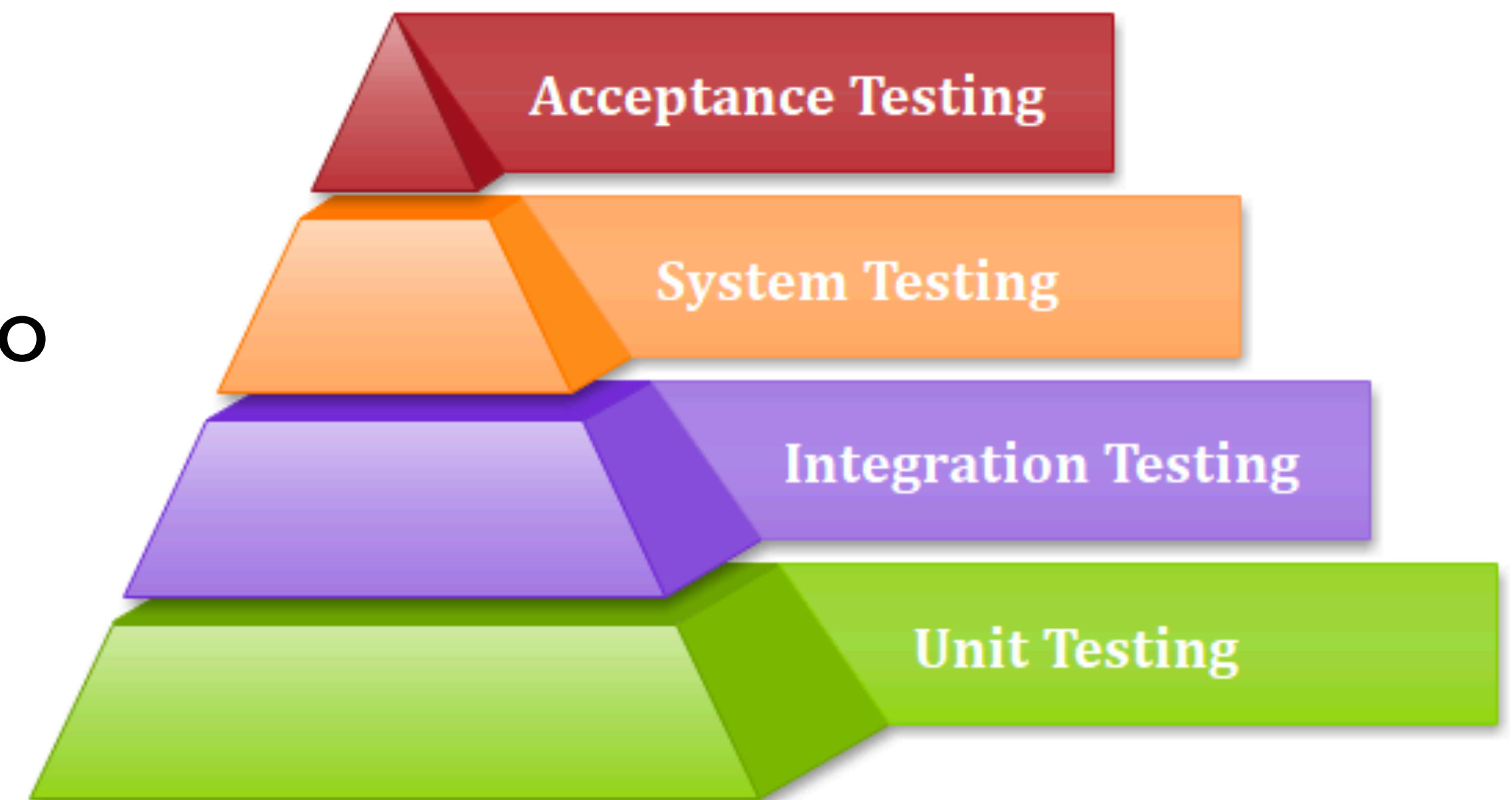


LIVELLI DEL TESTING

LIVELLI DEL TESTING



- **Test dei componenti (o d'unità):** test di singole unità del software
- **Test d'integrazione:** Integra diversi componenti tra loro per rilevare problemi d'interazione
- **Test di sistema:** il software è testato dopo che tutti i componenti sono stati integrati
- **Test di validazione:** condotti dall'utente finale per validare i requisiti





- Il test di unità consiste nel testare i singoli componenti del sistema in isolamento
- Ciascuna unità viene verificata sulla base della propria documentazione
- Avviene spesso durante la fase di implementazione
- Le unità da testare possono essere:
 - Metodi di una classe
 - Classi
 - Componenti riusabili che offrono una specifica interfaccia per accedere alle loro funzionalità



"Unit-level testing of methods and classes should be designed and performed by the class developer at the same time as the class development itself"

[Erich Gamma and Kent Beck (A.K.A. the authors of design patterns and eXtreme Programming, respectively)]

- Il testing da parte degli sviluppatori ha enormi vantaggi poiché essi hanno conoscenza del comportamento atteso, le precondizioni e le postcondizioni
- Gli sviluppatori tendono a difendere il proprio lavoro e a trovare meno difetti di tester indipendenti



- Il testing delle unità può essere oneroso, pertanto hanno molto successo le strategie di automatizzazione
- Il modo più banale di testare una classe e i suoi metodi é il **main-based approach**
- Un test method con il codice per esercitare le funzionalità é inserito in ogni unità
- **Problemi:**
 - ⦿ Il main aggiuntivo sarà anche distribuito nel prodotto finale
 - ⦿ Per testare comportamenti alternativi saranno necessari più main methods
- Una soluzione a tali problemi è usare le soluzioni X-Unit (come JUnit) che separano il codice di test dal codice sorgente

TESTING DI UNITÀ MAIN-BASED: ESEMPIO



```
package calcolatrice;
```

```
public class Calcolatrice {
```

```
    public int somma(int a, int b) {  
        return a+b;  
    }
```

```
}
```

```
package calcolatrice;
```

```
public class Main1 {
```

```
    public static void main(String[] args) {
```

```
        Calcolatrice c=new Calcolatrice();
```

```
        int somma=c.somma(3, 39);
```

```
        int atteso=42;
```

```
        if (somma==atteso)
```

```
            System.out.println("OK");
```

```
        else
```

```
            System.out.println("Failure");
```

```
    }
```

```
}
```

TESTING DI INTEGRAZIONE

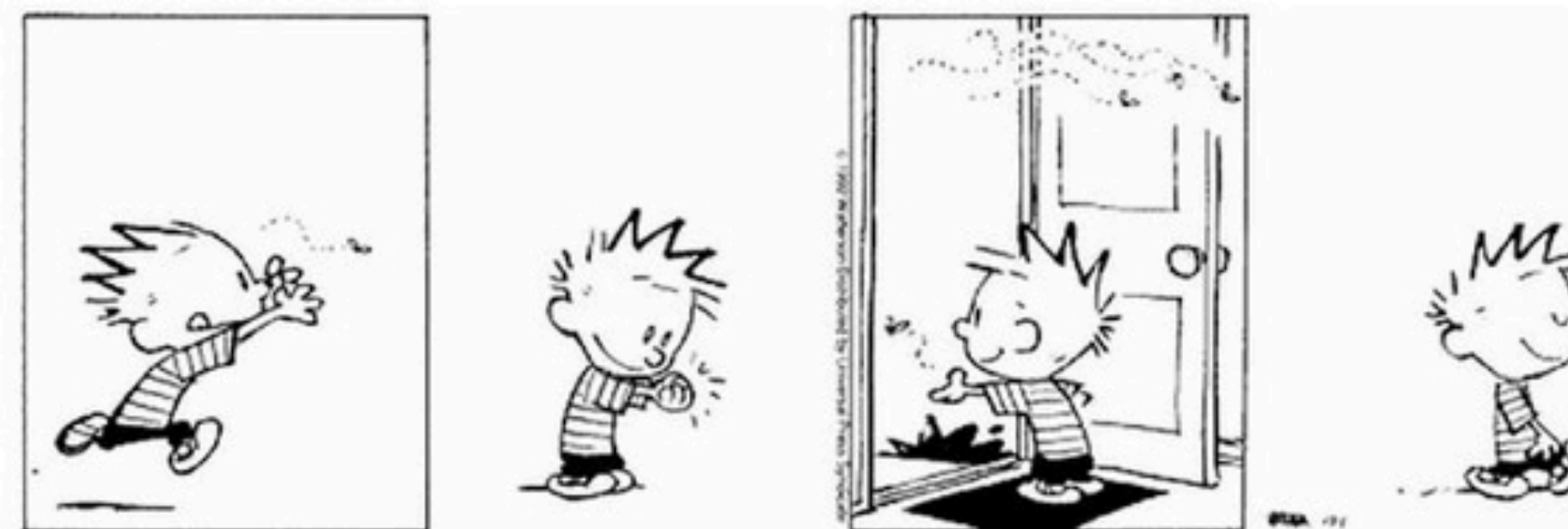


- Richiede la costruzione incrementale della struttura del sistema a partire dai componenti testati singolarmente
- Anche se i componenti in isolamento sono ritenuti corretti, bisogna verificare se ci sono malfunzionamenti durante la comunicazione tra i moduli, dovuti ad esempio a problemi nelle interfacce
- Viene testato il sottosistema ottenuto attraverso l'integrazione di gruppi di componenti che realizzano una determinata funzionalità



- Possibilità che dopo una modifica il software sia “regredito”, ossia che la modifica abbia introdotto difetti che prima non c'erano
- Dopo ciascuna integrazione, sarà necessario rieseguire i test precedenti per verificare che non ci siano nuovi malfunzionamenti del software precedentemente testato

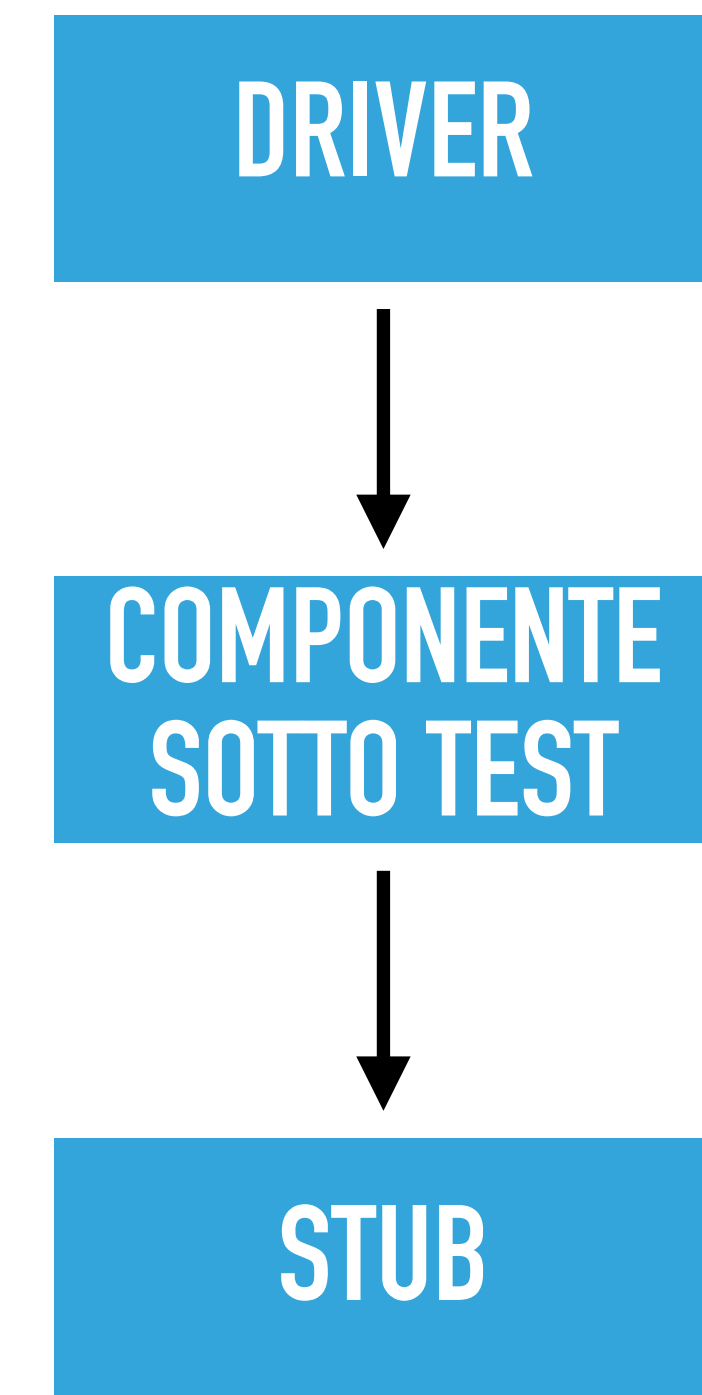
Regression:
"when you fix one bug, you
introduce several newer bugs."



TESTING DI INTEGRAZIONE: STUB E DRIVER



- In assenza del sistema completo sono fondamentali gli stub e i driver
- I moduli guida (**driver**) simulano i componenti chiamanti
 - Accettano i dati dei test case, invocano il componente chiamato e infine stampano i risultati
- I moduli fittizi (**stub**) simulano i componenti gerarchicamente inferiori al componente da testare
 - Sono invocati dal componente testato, simulano il funzionamento del componente chiamato
 - Funzioni fittizie, la cui correttezza è vera per ipotesi
- Driver e stub sono un overhead in quanto devono essere sviluppati ma non fanno parte del prodotto finale



STUB E DRIVER: ESEMPIO



- Vogliamo testare la funzione `prod_scalare(v1, v2)` senza coinvolgere gli altri componenti che la utilizzano
- `prod_scalare(v1, v2)` invoca la funzione `prodotto(a,b)` ma non abbiamo ancora realizzato quest'ultima pur conoscendone la specifica
- Nel metodo **driver** scriviamo il codice per eseguire alcuni casi di test:

```
main(){  
  p = prod_scalare([2,4],[4,7]);  
  return p;  
}
```

- Sostituiamo a `prodotto` il suo metodo **stub**

```
prodotto(a,b) {  
  if (a==2 && b==4) return 8;  
  if (a==4 && b==7) return 28;  
}
```



- Il software è testato dopo essere stato integrato (anche con altri elementi del sistema come hardware e dati)
- Richiede che l'integrazione sia stata completata
- Scopo del test è la verifica che tutti gli elementi del sistema siano stati correttamente integrati e che il sistema complessivo soddisfi i requisiti
- I test sono definiti sulla base della specifica del sistema



- Test di accettazione condotti dall'utente finale per consentire la validazione di tutti i requisiti
 - ◎ **α-Test:** condotto da un team selezionato composto da clienti e/o utilizzatori finali. Avviene in ambiente controllato, presso lo sviluppatore
 - ◎ **β-Test:** condotto da un numero maggiore di utenti finali con il proprio hardware specifico che possono avere interazioni non previste con il software. Non è presente, solitamente, lo sviluppatore. Può rivelare nuovi difetti o incompatibilità, ma è anche una forma di commercializzazione